

バーPOS 技術仕様書

開発者・システム担当者向けの仕様書です。計算ロジック、データ構造、基盤、セキュリティ、既知の制約までを記載します。

1. システム概要

iPad のブラウザ上で動作するバー向け POS（注文・会計・売上管理）です。売上データは Firebase Firestore にリアルタイム保存され、オーナーは別端末（PC等）で同じ URL を開くだけで最新の売上を確認できます。

- 注文入力（席ごと、品目単位で担当キャストを設定。席と未会計注文は Firestore に保存され、リロードしても復元・端末間で共有）
 - 卓は「+追加」または**定型ボタン**（よく使う卓名）で開く。各卓の「X」で閉じられ、**会計完了で自動削除。自動生成（A/B/C等）は行わない**。
 - 「フリー入力」タブで自由品目+**割引**を入力できる。
- 会計（現金 / カード / QR、**受取現金のフリー入力**+お釣り計算、決済手数料・実入金額の自動計算）。会計タブは常時開け、未会計の卓をチップで切替
- 入力日の表示・変更（遡及入力）。会計はこの日付で記録
- 売上管理（期間集計、支払方法別、キャストバック集計、CSV出力）
- **会計済み伝票の閲覧・編集・削除**（オーナー限定。編集は「注文入力の卓に戻して作り直す」方式。締め済み日はブロック）
- **日払い・大入の記録**（オーナー限定。キャスト別。レジ締めの残現金計算に反映）
- レジ締め（オーナー限定。日次の集計表示・締め確定で当日の入力をロック・締め解除も可）
- **打刻（勤怠）**：右上「打刻」ボタンで出勤/退勤を記録（オーナー・スタッフ共通）。**打刻管理**タブ（オーナー限定）で勤務のペアリング集計・不整合検出・修正・勤務CSV
- メニュー / キャスト / 手数料 / バック率 / 消費税をオーナーが画面から管理（キャストは**ニックネーム+本名**）
- バックは2系統: 卓バック（合計×卓バック率→卓の担当キャスト・複数なら頭割り）+ キャストドリンクバック（ドリンク料金×率→その担当へ上乗せ）
- 消費税は「税込で登録・加算しない」/「税抜で登録・会計時に加算」を切替+税率設定可
- スタッフ / オーナーの権限分離（ログイン）

2. 技術スタック・基盤

項目	内容
フロントエンド	React 18 + TypeScript + Vite 5
状態管理	Zustand 4
認証	Firebase Authentication（メール/パスワード）
データベース	Firebase Firestore
ホスティング	Firebase Hosting
対象端末	iPad Safari（PC ブラウザでも動作）
アイコン	Tabler Icons（CDN 読み込み）

- Firebase プロジェクト ID: `bar-pos-b1993`（本番）
- 公開 URL（デプロイ後）: `https://bar-pos-b1993.web.app`
- すべてクライアントサイド SPA。サーバーサイドのコードは無し（バックエンドは Firebase の各サービス）。
- 開発は Firebase エミュレータ（ローカル）で本番に触れずに行える（§9）。将来のパッケージ化方針（店ごとに別プロジェクト=A方式、dev/staging/prod）は [パッケージ化設計.md](#) を参照。

構成イメージ

```
[iPad Safari] ─┐
                │└─> Firebase Hosting（静的SPA配信）
[PC ブラウザ] ─┐
                │
                │└─> Firebase Authentication（ログイン／ロール判定）
                └─> Firestore（メニュー/キャスト/設定/席・注文/取引をリアルタイム同期）
```

3. ディレクトリ構成

```
src/
├─ components/
│   ├─ LoginScreen.tsx      # ログイン画面
│   ├─ OrderScreen.tsx      # 注文入力（席・メニュー・伝票）
│   ├─ CheckoutScreen.tsx   # 会計
│   ├─ SalesScreen.tsx      # 売上管理・取引明細・日払い/大入・レジ締め（オーナー専用）
│   ├─ MenuManageScreen.tsx # メニュー管理（オーナー専用）
│   ├─ CastManageScreen.tsx # キャスト管理（ニックネーム+本名。オーナー専用）
│   ├─ PunchModal.tsx      # 打刻（出勤/退勤）モーダル（全ユーザー）
│   └─ PunchManageScreen.tsx # 打刻管理（勤務集計・修正・勤務CSV。オーナー専用）
├─ hooks/
│   └─ useSalesSummary.ts    # 売上・バック集計ロジック
├─ lib/
│   ├─ firebase.ts          # Firebase 初期化（Firestore + Auth）・エミュレータ接続・コレクション定数
│   ├─ authConfig.ts        # ログインID⇄メール変換・ロール判定
│   ├─ tax.ts               # 税・手数料の計算ユーティリティ
│   ├─ punch.ts             # 打刻のペアリング・勤務時間算出
│   ├─ cast.ts              # キャスト表示名（店内=ニックネーム / CSV=本名）
│   ├─ csv.ts               # CSV 生成・ダウンロード（売上/バック/勤務）
│   └─ defaultMenus.ts      # デフォルトメニュー / フリー入力プリセット
├─ store/
│   └─ posStore.ts          # Zustand グローバル状態・全アクション
├─ types/index.ts           # 型定義
├─ styles/global.css        # スタイル
├─ App.tsx                 # ルート（認証ゲート・画面切替・購読）
├─ main.tsx                # エントリーポイント
├─ scripts/seed-emulator.mjs # エミュレータへサンプル投入（開発用）
├─ firestore.rules          # Firestore セキュリティルール
├─ firestore.indexes.json   # Firestore インデックス定義
├─ firebase.json            # Hosting (index.html は no-cache) + firestore + emulators 設定
└─ .env.emulator            # エミュレータ用のダミー設定 (VITE_USE_EMULATOR=true)
```

4. データモデル (Firestore)

コレクション名は `src/lib/firebase.ts` の `COLLECTIONS` で定義。

transactions (会計確定済みの取引)

会計確定時に1件追加。**オーナーは売上管理「取引明細」から編集（＝注文入力の卓に復元して作り直し）・削除が可能**（締め済み日はブロック）。スタッフは create のみ。

フィールド	型	説明
seatName	string	席名（未入力時は 席 <席ID> ）
solo	boolean	一人客フラグ
items	OrderItem[]	注文品目の配列（各品目に担当キャスト名を保持）
subtotal	number	税抜合計
tax	number	消費税額
total	number	税込合計
payMethod	'cash' 'card' 'qr'	支払方法
feeRate	number	決済手数料率（%）
feeAmount	number	決済手数料額
netAmount	number	実入金額（total – feeAmount）
primaryCast	string	主担当キャスト（売上最大の担当。CSV表示用）
tableCasts	string[]	卓バックの受取キャスト（卓の担当。複数なら頭割り）
completedAt	number	会計時刻（Unix ミリ秒）
_createdAt	Timestamp	サーバータイムスタンプ（書き込み時刻）

OrderItem（items 配列の要素）：

フィールド	型	説明
id	string	一意ID
name	string	品名
priceExTax	number	単価（税込モード時はその額が最終価格）
qty	number	数量
cast	string	担当キャスト名（キャストドリンクのドリンクバック用。空文字=未設定）
category	string	カテゴリ（キャストドリンク判定等。フリー入力は 'フリー入力' ）
isToday	boolean	本日限定メニュー
isFree	boolean	フリー入力

menus（メニュー）

{ name, priceExTax, category, isToday, sortOrder }。sortOrder 昇順で表示。本日限定は isToday: true ・ category: '本日限定'。カテゴリは MENU_CATEGORIES（src/lib/defaultMenus.ts）：セット / ショット / シャンパン / キャストドリンク / ウイスキー / カクテル / ビール / フード。

casts（キャスト）

{ name, realName?, sortOrder }。name =ニックネーム（源氏名。店内表示）、realName =本名（給与・CSV。空ならニックネームで代替）。sortOrder 昇順で表示。注文画面の担当選択肢・打刻の選択肢になる。

tables（席・未会計の注文。ドキュメントID=席ID）

{ name, solo, tableCasts: string[], items: OrderItem[], createdAt, updatedAt }。tableCasts は卓の担当キャスト（複数可）。

- 注文操作のたびに該当席のドキュメントを保存（ローカル楽観更新+Firestore書き込み）。
- ログイン後に購読してハイドレートするため、リロードしても復元・端末間で共有。
- 卓は「+追加」または定型ボタンで作成（addSeat）。会計完了・「×」で該当ドキュメントを削除**（席バーから消える）。
- 自動生成は一切行わない**（旧仕様の「空なら席 A/B/C を作成」は廃止）。
 - △ 注意: 旧ビルドがこの自動生成を持つ。旧ビルドの端末が1台でも開いていると、tables を空にするたびに A/B/C を書き戻す。全端末を新ビルドに更新（開き直し）すること。対策として index.html を no-cache 化済み (§11)。

punches (打刻イベント。ドキュメント自動ID)

{ castId, name, realName?, type: 'in'|'out', at: number, date: 'YYYY-MM-DD', by?: 'owner'|'staff' }。出勤/退勤を1イベント=1ドキュメントで記録。勤務（出勤～退勤のペア）は保存せず、src/lib/punch.ts の buildShifts が時系列でペアリングして算出（日付をまたぐ勤務は出勤日に帰属）。

payouts (日払い・大入。ドキュメント自動ID)

{ castId, name, type: '日払い'|'大入', amount: number, date: 'YYYY-MM-DD', createdAt }。レジ締めめの「金庫に残る現金」計算に反映。

settings (設定。ドキュメントIDで区別)

- settings/fees ... { card: number, qr: number } (各%)
- settings/back ... { rate: number, drinkRate: number } (卓バック率・キャストドリンクバック率。0.10 = 10%)
- settings/tax ... { rate: number, mode: 'exclusive' | 'inclusive' } (消費税率と税の扱い)

closures (レジ締め。ドキュメントID=YYYY-MM-DD)

{ date, closedAt, totalSales, cash, card, qr, totalFee, totalNet, totalBack, txCount }。ドキュメントが存在する日付＝締め済み。締め済みの日付には会計を記録できない（completePayment でガード+会計画面で無効化）。締め解除はドキュメント削除。

会計の completedAt は入力日（entryDate、既定は今日。遡及入力で過去日も可）から生成する。
entryDate は端末の状態でリロードすると今日に戻る。表示中の画面・選択中の席は localStorage に保存して復元する。

5. 認証・権限

ログイン方式

Firebase Authentication の「メール/パスワード」を使用。現場で打ちやすいよう、ログイン画面では **ユーザーID** だけを入力し、内部でメールアドレスへ変換する（src/lib/authConfig.ts）。

- 変換: ID → ID@<VITE_AUTH_ID_DOMAIN> (既定ドメイン bar-pos.local)
- 例: ログインID owner → owner@bar-pos.local

ロール判定

ログイン中のメールアドレスで自動判定（DB にロールを持たせない）。

```
role = (email === `${VITE_OWNER_ID}@${VITE_AUTH_ID_DOMAIN}`) ? 'owner' : 'staff'
```

- `owner@bar-pos.local` → **オーナー**（全画面）
- それ以外 → **スタッフ**（注文入力・会計のみ）

アプリ側（画面の出し分け）と Firestore ルール（DB アクセス制御）の **両方** が同じメールで判定するため、UI を隠すだけでなく DB レベルでも保護される。

Firestore セキュリティルール（`firestore.rules`）

コレクション	read	write
<code>menus</code>	ログイン済み	オーナーのみ
<code>casts</code>	ログイン済み	オーナーのみ
<code>settings</code>	ログイン済み	オーナーのみ
<code>tables</code>	ログイン済み	ログイン済み（スタッフも注文を取るため読み書き可）
<code>closures</code>	ログイン済み	オーナーのみ（レジ締め・解除）
<code>transactions</code>	オーナーのみ	create はログイン済み（＝スタッフも会計確定可）、update/delete はオーナーのみ
<code>punches</code>	ログイン済み	create はログイン済み（＝スタッフも打刻可）、update/delete はオーナーのみ
<code>payouts</code>	オーナーのみ	オーナーのみ

→ スタッフのアカウントでは（UI だけでなく）DB から直接でも過去売上・日払いを読めない。打刻は打てる（create）が、修正・削除はオーナーのみ。

ルール内のオーナー判定は `owner@bar-pos.local` をハードコードしている。`VITE_OWNER_ID` / `VITE_AUTH_ID_DOMAIN` を変えたら、ルールの該当箇所も `<VITE_OWNER_ID>@<VITE_AUTH_ID_DOMAIN>` に合わせること。

6. 計算ロジック

実装は `src/lib/tax.ts` (税・手数料)、`src/store/posStore.ts` (会計確定)、`src/hooks/useSalesSummary.ts` (集計)。

6.1 消費税 (税率・税の扱いは設定可能)

消費税率 (`settings/tax.rate`、既定0.10) と「税の扱い」 (`settings/tax.mode`) をオーナーが画面から設定する。実装は `calcBill(base, rate, mode)` (`src/lib/tax.ts`)。 $base = \sum(priceExTax \times qty)$ 。

exclusive (税抜で登録・会計時に加算) 明細は税抜で表示し、消費税は小計に対して1回だけ計算 (端数切り捨て)。「明細の合計=小計=合計欄」が一致する。

```
subtotal = base
tax       = floor(base × rate)
total     = subtotal + tax
```

メニューボタンの単価表示は税込 (`displayUnit = floor(price × (1+rate))`) = お客様向け総額表示。

inclusive (税込で登録・加算しない) 登録額がそのまま最終価格。消費税は加算も表示もしない。

```
subtotal = base
tax       = 0
total     = base
```

この場合、メニュー単価表示は登録額そのまま (`displayUnit = price`)。注文/会計画面の小計・消費税行は非表示。

例 (exclusive・税率10%) : 税抜 545 円 × 3 → subtotal 1,635 / tax 163 / total 1,798 例 (inclusive) : 税込 599 円 × 3 → total 1,797 (消費税の行は出さない)

6.2 決済手数料・実入金額

```
feeRate   = (cash: 0, card: settings.fees.card, qr: settings.fees.qr) // %
feeAmount = floor(total × feeRate / 100)
netAmount = total - feeAmount
```

既定値: カード 3.25%、QR 1.98% (オーナーが画面から変更可)。現金は手数料 0。

例: total 10,000・カード 3.25%


```
feeAmount = floor(10,000 × 0.0325) = 325
netAmount = 9,675
```

6.3 お釣り（現金）

```
change = cashReceived - total // cashReceived ≥ total のときのみ確定可能
```

預かり金額は `total` 以上のプリセット（1,000/2,000/5,000/10,000+ぴったり額）から選択。

6.4 キャストバック（2系統：卓バック+キャストドリンクバック）

実装は `useSalesSummary`。設定は `settings/back` の `rate`（卓バック率）と `drinkRate`（ドリンクバック率）。

① **卓バック**：会計の合計に卓バック率をかけ、卓の担当キャスト（`tableCasts`）で**頭割り**。

```
n = tableCasts.length
各 tableCast へ: backRaw += (total × rate) / n
（担当が0人の卓はバック対象外＝未設定）
```

② **キャストドリンクバック**：`category === 'キャストドリンク'` の品目について、料金 × ドリンク率を**その品目の担当**（`item.cast`）へ上乗せ。

```
そのドリンクの担当へ: backRaw += (priceExTax × qty) × drinkRate
```

- キャストドリンクの料金は **卓バックの合計にも含まれる** ため、卓の担当とドリンクの担当の**両方**に効く（二重計上＝仕様）。
- 最後に各キャストの `backRaw` を四捨五入して `backAmount`。
- 担当未設定はバック対象外（0円）。集計には可視化。

例（卓バック10% / ドリンクバック率はお店の設定。卓の担当=たか子）：

```
セット 2,000 + キャストドリンク（ゆう子）1,200 = 合計3,200
→ たか子（卓バック）：3,200 × 10% = 320
→ ゆう子（ドリンクバック）：1,200 × ドリンク率
卓の担当が2名なら卓バックは頭割り（320 → 160ずつ）
```

6.5 売上サマリー（`useSalesSummary`）

期間内の `transactions` から集計:

```

totalSales      = Σ total          // 税込
totalFee        = Σ feeAmount
totalNet        = totalSales - totalFee
txCount         = 取引件数
avgPerCustomer  = round(totalSales / txCount) // 1取引あたり（人数ベースではない）
soloCount       = solo の件数
byMethod[m]     = { count, sales(=Σtotal), fee, net } を支払方法別に集計
castSummaries   = 6.4 の結果（卓数 / 卓売上+ドリンク売上 / 卓バック+ドリンクバック）

```

注: キャスト別「売上」は卓担当の卓売上+担当ドリンク料金で、キャストドリンクは卓担当・ドリンク担当に二重計上され得るため、上部「売上合計」とは一致しない（仕様）。

6.6 営業日の定義（17:00始まり）と期間の範囲

営業日は 17:00～翌17:00 を1日とする（`BUSINESS_DAY_START_HOUR = 17`）。深夜帯（翌0:00～16:59）の売上・打刻は**前日の営業日**として集計される。実装は `dateStrOf(ts)` が「タイムスタンプから17hを引いてから暦日を取る」ことで実現（売上・打刻の日付判定はすべてこの関数を経由）。

- `businessDayStart(dateStr)` = その営業日の 17:00、`businessDayEnd(dateStr)` = 翌営業日の開始（翌日17:00）。範囲は `[start, end)`。
- 例: `completedAt = 7/4 02:00` の取引 → 営業日 **7/3**。 `7/4 17:00` の取引 → 営業日 **7/4**。

期間の範囲（`SalesScreen.periodRange`、`entryDate`=基準営業日）：

期間	from	to
今日	基準営業日 17:00	翌営業日 17:00 の直前
今週	6営業日前 の 17:00（直近7営業日）	同上
今月	当月1日（営業日） 17:00	同上

- 「今週」はカレンダー一週ではなく直近7営業日。
- この定義により、**深夜の会計がレジ締め日とズれる**問題も解消（`closures` は営業日で締めるため `dateStrOf(completedAt)` と一致する）。
- 17:00 の境界は定数。将来 `settings` で店舗ごとに可変化する余地あり（[パッケージ化設計.md](#)）。

7. リアルタイム同期

`onSnapshot` で購読:

- `menus` ・ `casts` ・ `tables` ・ `closures` : ログイン後に全ユーザーが購読。`tables` はハイドレートして席・注文を復元（リロード耐性・端末間共有）。`closures` は締め済み日付の一覧として保持し、会計のブロック判定に使う。
- `transactions` : 売上画面（オーナー）で、選択期間を `where(completedAt >= from && <= to)` で絞り込み購読（全件取得はしない）。
- `punches` : 打刻モジュール（当日）・打刻管理（検索期間）で `where` 範囲購読。日付跨ぎのペアリングのため前後に約±12h のバッファを持って取得。
- `payouts` : 売上画面（オーナー）で選択期間を `where(date >= from && <= to)` で購読。

注文操作はローカル楽観更新+該当 `tables` ドキュメントへ書き込み（fire-and-forget）。会計確定は `transactions` へ `addDoc`（追記）し、その席の `items` を空にする。表示中の画面と選択中の席は **localStorage** に保存し、リロード後に同じ画面・同じ席へ復元する（端末ローカルのUI状態）。

8. CSV 出力（`src/lib/csv.ts`）

- BOM 付き UTF-8（Excel で文字化けせず開ける）。
- 売上CSV（`buildTransactionCSV`）：日時/席名/一人客/支払方法/税抜売上/消費税/税込売上/手数料率/手数料額/実入金額/主担当キャスト。
- バックCSV（`buildCastCSV`）：キャスト/担当件数/売上合計(税抜)/バック額。
- 勤務CSV（`buildShiftCSV`。打刻管理から出力）：キャスト（本名）/出勤/退勤/勤務時間。

9. 環境変数（`.env`）

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_AUTH_ID_DOMAIN=bar-pos.local    # ログインID→メール変換ドメイン
VITE_OWNER_ID=owner                 # オーナー扱いにするログインID
VITE_BACK_RATE=0.30                  # バック率の初期値（画面保存値が優先）
```

`.env` は `.gitignore` 済み（Firebase キーを含むためリポジトリに含めない）。

開発用エミュレータ（`.env.emulator`。コミット可＝ダミー値のみ）

`VITE_USE_EMULATOR=true` のとき、`src/lib/firebase.ts` がローカルの Firestore/Auth エミュレータに接続する（`demo-bar-pos`）。本番ビルドはこのフラグを持たないため接続しない。→ 開発・自動検証で本番データを汚さない。詳細は [パッケージ化設計.md](#)。

```
npm run emu          # エミュレータ起動（要 Java。UI: http://127.0.0.1:4000）
npm run seed:emu     # サンプル（アカウント/メニュー/キャスト/設定）投入
npm run dev:emu      # アプリをエミュレータ接続で起動（vite --mode emulator）
```

10. 既知の制約・注意点

実運用前に把握しておくべき項目（現時点で未対応）。

1. **専用の返金フローは無い**。オーナーは「取引明細」から会計済み伝票の削除・編集（注文入力に戻して作り直し）が可能だが、返金額・理由の記録などの返金台帳機能は無い。
2. **オフライン耐性が限定的**。会計確定時にネットワーク失敗してもアプリ上に明確なエラー表示が出ない（Firestore SDK のキュー任せ）。Firestore のオフライン永続化（`persistentLocalCache`）も未設定のため、完全オフラインでのリロードは未保証。
3. **CSV インジェクション対策なし**。席名等に `=`, `+`, `-`, `@` で始まる文字列を入れると、Excel で数式として解釈され得る。
4. **「今週」は直近7日**（カレンダー週ではない）。
5. **客単価は1取引あたり**（来店人数ベースではない）。
6. **キャストのリネーム/削除は過去データに遡及しない**。取引内には会計時点のキャスト名（文字列）が保存されるため、改名後も過去記録は旧名のまま。
7. **席・注文の競合**：同じ卓を2端末で同時編集した場合は最後の書き込みが優先（小規模運用を想定）。

補足：「席・未会計の注文がリロードで消える」問題は `tables` コレクションへの永続化で解消済み（オンライン時）。

11. セットアップ・デプロイ

詳細は [README.md](#) を参照。要点のみ：

```
npm install
cp .env.example .env    # Firebase 設定値とログイン設定を記入
npm run dev              # http://localhost:5173

# デプロイ
npm run build
firebase deploy          # Hosting (firebase.json 設定済み)
```

- Firestore ルールは `firebase deploy --only firestore:rules` で反映できる（`firebase.json` に `firestore` 設定済み）。または Firebase Console → Firestore → ルール に `firestore.rules` を貼って「公開」。
- Hosting の `index.html` は no-cache（`firebase.json` の headers）。旧ビルドのキャッシュ残留による不整合（§4 tables の A/B/C 書き戻し等）を防ぐため。デプロイ後は各端末を開き直すこと。
- Authentication で「メール/パスワード」を有効化し、`owner@bar-pos.local` / `staff@bar-pos.local` を作成。
- メニュー・キャストの初期データは、オーナーでログインして各管理画面の「投入」ボタンから登録。

12. 今後の改善候補

- 取引の取消・返金フロー
- オフライン永続化（`persistentLocalCache`）と会計失敗時のエラー表示
- CSV インジェクション対策（先頭エスケープ）
- 真の PWA 化（manifest / Service Worker）